

MICROBOT ECOSYSTEM

Documento 03 dei 12

Hardware Architecture Specification

Architettura elettronica, alimentazione, pinout, sicurezza e integrazione dei sette prototipi

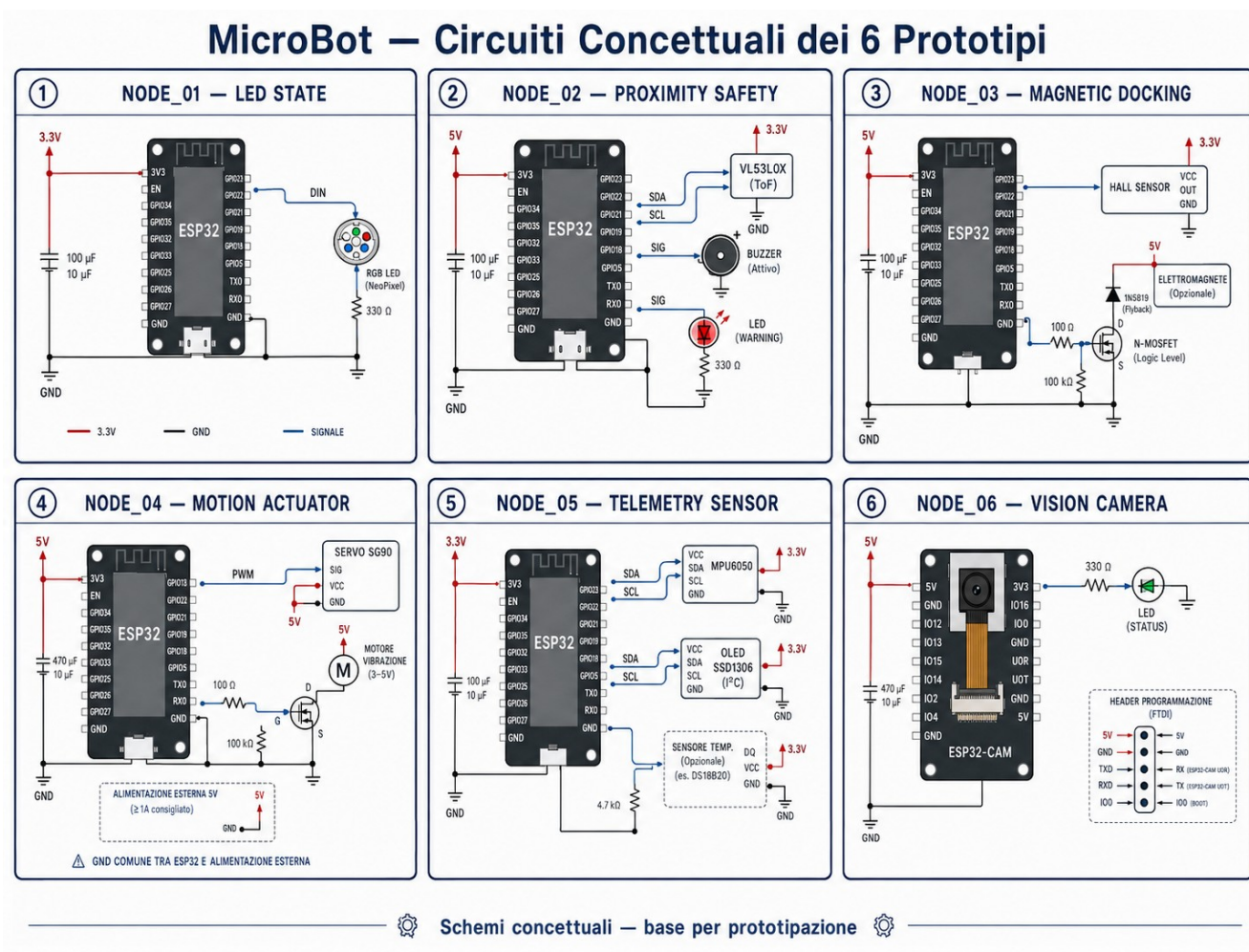


Figura 1 - Vista concettuale dei circuiti dei sei prototipi MicroBot.

Versione: v0.1 | Autore progetto: Anton | Uso: progettazione, costruzione e validazione hardware MicroBot

1. Scopo del documento

Questo documento definisce l'architettura hardware di MicroBot Ecosystem nella sua versione prototipale v0.1. Il suo obiettivo è trasformare il materiale teorico e operativo del progetto in una specifica elettronica concreta: quali blocchi hardware esistono, come comunicano, come vengono alimentati, quali pin usare, quali protezioni inserire e come collegare Master, nodi, dashboard e simulazione.

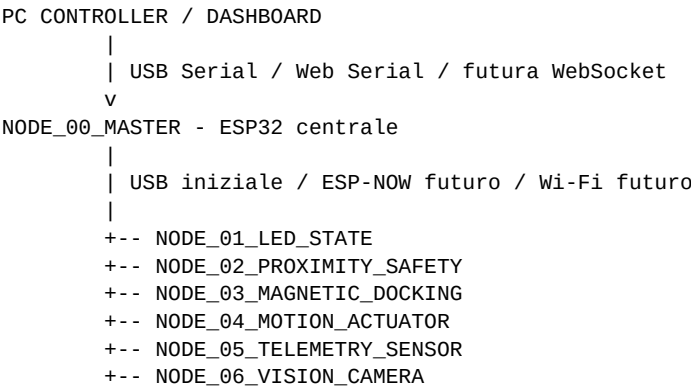
Il documento non sostituisce gli schemi elettrici definitivi PCB, ma stabilisce una base ingegneristica coerente per costruire la prima piattaforma reale: un controller PC, un Master ESP32 e sei nodi specializzati. Questa scelta deriva dal manuale operativo dei prototipi, che definisce una prima architettura fisica composta da un nodo Master, sei nodi prototipo e una logica di comunicazione comune.

Principio guida

Prima si costruisce una piattaforma robusta, leggibile e debuggabile; solo dopo si miniaturizza. Per questo motivo la v0.1 usa breadboard, moduli ESP32 e sensori commerciali, mentre il design MicroBot nero e compatto rimane obiettivo di industrial design e mockup fisico.

2. Architettura hardware generale

L'architettura hardware MicroBot v0.1 è organizzata secondo un modello Master / Node. Il Master ESP32 coordina il sistema, riceve comandi dal PC, inoltra comandi ai nodi e raccoglie stati, errori e telemetria. Ogni nodo possiede un'identità, una funzione specifica, uno stato interno e un set ridotto di comandi. Questa impostazione consente di validare singole funzioni senza dover costruire immediatamente micro-robot autonomi completi.



Blocco	Ruolo	Hardware base
PC Controller	Dashboard, serial bridge, simulazione, log, interfaccia utente	Computer/Mac + browser o Python/Node bridge
NODE_00_MASTER	Orchestrazione, scan, stato nodi, comandi, emergency stop	ESP32 DevKit + OLED + LED + pulsanti
NODE_01	Stato visivo/simbolico dello swarm	ESP32 + LED RGB / NeoPixel
NODE_02	Distanza, sicurezza, allarme prossimità	ESP32 + VL53L0X o HC-SR04 + buzzer
NODE_03	Docking magnetico e rilevamento aggancio	ESP32 + Hall sensor + magneti + MOSFET opzionale
NODE_04	Movimento, servo, vibrazione	ESP32 + servo SG90 + motorino vibrazione
NODE_05	Telemetria, IMU, display locale	ESP32 + MPU6050 + OLED SSD1306
NODE_06	Visione, stream camera, camera status	ESP32-CAM + FTDI + alimentazione stabile

3. Principi di progettazione hardware

Principio	Descrizione
Modularità	Ogni nodo deve essere costruibile, testabile e sostituibile separatamente.
Debug prima della miniaturizzazione	La prima fase usa breadboard e moduli commerciali per rendere visibili errori, tensioni e cablaggi.
Separazione logica/attuazione	L'ESP32 genera segnali logici a 3.3 V; attuatori, motori e bobine devono usare driver e alimentazione adeguata.
GND comune	Quando si usa alimentazione esterna, la massa dell'alimentatore deve essere collegata alla massa dell'ESP32.
Safe state	Ogni nodo deve poter andare in STOP, ERROR o LOW_POWER senza lasciare attuatori attivi.
Compatibilità futura	I pinout devono essere abbastanza coerenti da facilitare il passaggio da breadboard a millefori e poi PCB.

4. Alimentazione e distribuzione della potenza

L'alimentazione è uno dei punti più delicati dell'intero sistema. L'ESP32 lavora a logica 3.3 V, mentre molti attuatori e sensori possono richiedere 5 V o correnti superiori a quelle disponibili dai pin del microcontrollore. Per la v0.1 si consiglia di alimentare i prototipi tramite USB o power bank, evitando batterie Li-ion/Li-Po nude finché non esiste un piano di protezione e ricarica corretto.

Carico	Alimentazione	Criticità	Regola progettuale
ESP32 DevKit	USB 5 V / regolatore onboard	Logica 3.3 V	Non alimentare carichi pesanti dal pin 3.3 V
Sensori I2C	3.3 V preferibile	Basso consumo	Verificare compatibilità logica 3.3 V
Servo SG90	5 V esterno	Picchi di corrente	GND comune obbligatorio
Motore vibrazione	3-5 V esterno	Corrente impulsiva	Usare MOSFET/transistor e diodo se necessario
Elettromagnete / coil	5 V o alimentazione dedicata	Corrente elevata e calore	MOSFET logic-level + flyback diode + duty cycle limitato
ESP32-CAM	5 V stabile	Sensibile a cadute tensione	Power supply robusto e cavi corti

Regola assoluta sui 5 V

I segnali a 5 V non devono entrare direttamente nei GPIO dell'ESP32. Se un sensore come HC-SR04 restituisce ECHO a 5 V, serve un partitore resistivo o un level shifter.

5. Mappa pinout standard consigliata

Per ridurre confusione tra i nodi, la v0.1 usa una mappa pinout standard. Non tutti i pin ESP32 sono equivalenti; alcuni sono pin di boot strap e possono creare problemi all'avvio. Se un pin crea instabilità, va cambiato e documentato nel registro hardware.

Funzione	Pin consigliato	Uso	Nota
I2C SDA	GPIO 21	OLED, MPU6050, VL53L0X	Bus condiviso, indirizzi diversi
I2C SCL	GPIO 22	OLED, MPU6050, VL53L0X	Bus condiviso
LED STATUS	GPIO 2	LED stato base	Attenzione: LED onboard su molte board

LED ERROR / COMM	GPIO 4	Errore/comunicazione	Usare resistenza 220 ohm
BUZZER	GPIO 15	Allarme NODE_02	Verificare boot strap se dà problemi
HC-SR04 TRIG	GPIO 5	Trigger sensore distanza	Output ESP32 ok
HC-SR04 ECHO	GPIO 18	Echo tramite partitore	Mai 5 V diretto
Hall A3144	GPIO 19	Rilevamento magneti	Input digitale
Servo signal	GPIO 13	PWM servo SG90	Alimentazione servo separata
Motor MOSFET gate	GPIO 25/26	Vibration motor / coil	Gate resistor + pulldown consigliati
Pulsante START	GPIO 32	Master	INPUT_PULLUP
Pulsante SCAN	GPIO 33	Master	INPUT_PULLUP

6. NODE_00_MASTER - Hardware architecture

Il Master è il controller fisico centrale. Non deve eseguire calcoli pesanti di simulazione; deve coordinare, inviare comandi, ricevere stati, gestire errori e inoltrare informazioni al PC. Nella versione iniziale il PC si collega via USB seriale al Master. In una versione successiva, il Master parla con i nodi tramite ESP-NOW.

Componente	Ruolo	Funzione
ESP32 DevKit	Unità centrale	Riceve comandi dal PC e coordina i nodi
OLED SSD1306	Display locale	Mostra stato Master, nodi online/offline, errori
LED verde	Power/ready	Master acceso e operativo
LED blu	Communication	Trasmissione/ricezione in corso
LED rosso	Error	Nodo offline, allarme o errore
Pulsante START/STOP	Controllo manuale	Avvio o arresto logico sistema
Pulsante SCAN/RESET	Diagnostica	Scan nodi o reset stato interno

Master boot flow:

1. inizializza Serial
2. inizializza OLED
3. configura LED e pulsanti
4. carica lista nodi attesi
5. invia PING / scan
6. aggiorna tabella nodi
7. accetta comandi PC
8. invia STATUS periodico al PC
9. entra in ERROR se un nodo critico non risponde
10. STOP globale in caso di allarme

7. NODE_01_LED_STATE - Stato visivo

Il nodo LED è il primo prototipo reale perché permette di validare comando, risposta, stato interno e sincronizzazione senza rischi elettrici elevati. Può usare tre LED separati o un LED RGB/NeoPixel. Per un primo test è sufficiente un LED RGB con resistenze adeguate o un piccolo modulo NeoPixel alimentato correttamente.

Stato	Significato	Output consigliato
IDLE	Stato base	Luce bassa o spento
ACTIVE	Nodo attivo	Colore/stato stabile
DOCKING	Modalità aggancio	Pattern dedicato
ERROR	Errore locale	Rosso o lampeggio rapido
CHARGING	Ricarica o alimentazione esterna	Giallo/bianco lento
SYNC	Sincronizzazione	Breve flash coordinato

8. NODE_02_PROXIMITY_SAFETY - Sensore distanza e allarme

Il nodo prossimità introduce percezione e sicurezza. La soluzione consigliata è VL53L0X perché usa I2C e lavora bene con logica 3.3 V. HC-SR04 può essere usato come alternativa economica, ma richiede attenzione sul pin ECHO, che in molte versioni lavora a 5 V.

Elemento	Collegamento	Nota
VL53L0X	SDA GPIO 21, SCL GPIO 22	Scelta consigliata; compatto e stabile
HC-SR04 TRIG	GPIO 5	Trigger digitale in uscita
HC-SR04 ECHO	GPIO 18 tramite partitore	Ridurre 5 V a circa 3.3 V
Buzzer	GPIO 15	Allarme acustico opzionale
LED warning	GPIO 4 o 2	Stato WARNING/DANGER

```
Partitore HC-SR04 ECHO:  
ECHO 5V -- R1 1k -- punto centrale -- GPIO ESP32  
                |  
                R2 2k  
                |  
                GND  
Risultato: circa 3.3 V sul GPIO.
```

9. NODE_03_MAGNETIC_DOCKING - Aggancio magnetico

Il nodo magnetico è il collegamento più diretto con la materia programmabile. Nella v0.1 non deve generare subito forze importanti: deve rilevare il magnete e dichiarare uno stato DOCKED stabile. La versione base usa sensore Hall e magneti permanenti. La versione avanzata aggiunge elettromagnete, MOSFET logic-level e diodo di flyback.

Elemento	Collegamento	Funzione
Sensore Hall A3144 / KY-003	GPIO 19	Rilevamento magnete vicino
Magneti al neodimio	Struttura fisica	Aggancio passivo e test docking
Elettromagnete	Carico separato	Solo fase avanzata
MOSFET N logic-level	Gate da GPIO con resistenza	Interruttore elettronico per coil
Diodo flyback	In parallelo alla bobina	Protegge da picchi induttivi
Pulldown gate	100k verso GND	Evita attivazioni casuali

Nota termica sulle bobine

Le coil possono scaldare rapidamente. Nella v0.1 usare duty cycle limitato, test brevi, alimentatore controllato e mai lasciare la bobina attiva senza supervisione.

10. NODE_04_MOTION_ACTUATOR - Movimento e attuazione

Il nodo movimento dimostra il passaggio da comando software ad azione fisica. La prima versione può usare un servo SG90 per mostrare rotazione controllata e un mini vibration motor per simulare vibrazione o risposta tattile. Il servo richiede alimentazione 5 V esterna o power bank; non deve essere alimentato dal pin 3.3 V dell'ESP32.

Elemento	Collegamento	Funzione
Servo SG90	Signal GPIO 13, VCC 5 V, GND comune	Rotazione controllata tramite PWM
Mini vibration motor	MOSFET + alimentazione 3-5 V	Vibrazione simulativa
Condensatore 470 uF	Tra 5 V e GND vicino al servo	Riduce cadute di tensione
STOP command	Firmware	Disattiva movimento in allarme

11. NODE_05_TELEMETRY_SENSOR - Telemetria

Il nodo telemetria rende il sistema osservabile. La combinazione consigliata è MPU6050 + OLED SSD1306 su bus I2C. Il nodo legge accelerazione, giroscopio e stato interno, poi invia al Master un pacchetto periodico. L'OLED può mostrare stato locale, IP, errore, allarme o messaggi del Master.

Elemento	Collegamento	Funzione
MPU6050	SDA GPIO 21, SCL GPIO 22	Accelerometro e giroscopio
OLED SSD1306	SDA GPIO 21, SCL GPIO 22	Display locale
Sensore temperatura opzionale	GPIO digitale/I2C/1-Wire	Monitoraggio termico
I2C scanner	Firmware test	Verifica indirizzi e cablaggio

Telemetry packet example:

```
{
  "from": "NODE_05_TELEMETRY_SENSOR",
  "accel": {"x": 0.02, "y": 0.01, "z": 9.81},
  "gyro": {"x": 0.2, "y": 0.0, "z": 0.1},
  "temperature": 26.4,
  "status": "OK"
}
```

12. NODE_06_VISION_CAMERA - Camera e percezione

Il nodo camera usa ESP32-CAM. Per la v0.1 è sufficiente che il nodo inizializzi la camera, esponga uno stream o invii il proprio IP, e risponda a PING/STATUS. Il Master non deve ricevere direttamente il video: è più stabile lasciare che la dashboard o il PC apra lo stream separatamente.

Elemento	Ruolo	Nota
ESP32-CAM	Modulo principale	Camera, Wi-Fi, stato visione
FTDI USB-seriale	Programmazione	Necessario per caricare firmware
GPIO0 a GND	Solo upload	Poi scollegare per boot normale
5 V stabile	Alimentazione	Fondamentale contro reset casuali
Supporto orientabile	Meccanico	Treppiede, cartone rigido o stampa 3D

13. Comunicazione fisica e rete

Il progetto prevede tre modalità progressive. La prima, USB/seriale, serve per debug. La seconda, ESP-NOW, è la più coerente con uno swarm MicroBot v0.1. La terza, Wi-Fi con dashboard web, è da introdurre solo dopo stabilizzazione del firmware base.

Fase	Tecnologia	Schema	Nota
Fase 1	USB / Seriale	PC collegato al Master o a singoli nodi	Debug facile, molti cavi
Fase 2	ESP-NOW	Master comunica wireless con nodi	Coerente con swarm; richiede MAC address
Fase 3	Wi-Fi + Dashboard	API, WebSocket, logging	Potente ma più complessa

Recommended communication evolution:

Fase 1: PC -> USB Serial -> Master

Fase 2: PC -> USB Serial -> Master -> ESP-NOW -> Nodes

Fase 3: Dashboard -> WebSocket/API -> Master -> ESP-NOW/Wi-Fi -> Nodes

14. Safety layer hardware

Il safety layer non è opzionale. Serve a proteggere ESP32, batterie, attuatori e utente. Ogni nodo deve avere una condizione sicura di default: attuatori spenti, bobine disattivate, LED/error state leggibile e risposta STOP sempre prioritaria.

Area	Rischio	Contromisura
GPIO ESP32	Sovratensione 5 V	Level shifter o partitore; verificare sensori
Servo/motori	Picchi corrente e reset ESP32	Alimentazione separata, GND comune, condensatore
Bobine	Surriscaldamento e picchi induttivi	MOSFET, flyback diode, PWM limitato, test brevi
Batterie	Cortocircuito, sovrascarica, carica non controllata	Power bank iniziale; Li-ion/LiPo solo con protezione
Firmware	Freeze o comando bloccato	Watchdog, timeout, STOP globale
Cablaggio	Inversioni e corti	Color coding, etichette, multimetro prima dell'accensione

15. Schema fisico di integrazione con il design MicroBot nero

Il design finale nero, composto da due coni tronchi e una sfera centrale, deve essere trattato come obiettivo meccanico e industrial design. Dal punto di vista hardware, la sfera centrale ospita microcontrollore, sensori e docking ring; le parti laterali possono contenere batteria, magneti, contatti e microattuatori. Nella v0.1 questa forma viene realizzata come mockup o shell, mentre i circuiti rimangono su prototipi più grandi e accessibili.



Figura 2 - Concept tecnico del MicroBot nero: guscio, sensore centrale, docking ring e parti modulari.

Parte fisica	Contenuto possibile	Funzione
Sfera centrale	ESP32 compatto / sensori / anello docking	Centro logico e percettivo
Cono superiore	Batteria o microattuazione	Distribuzione massa e simmetria
Cono inferiore	Magneti / contatti / batteria	Aggancio e stabilità
Anello centrale	Magneti / contatti spring / sensore Hall	Docking e riconoscimento unità vicine
Finestra sensore	IR/camera/finto sensore nella v0.1	Identità estetica e percezione

16. Roadmap hardware: da breadboard a PCB

La roadmap hardware deve procedere a livelli. Saltare direttamente al PCB o alla miniaturizzazione riduce la probabilità di successo. Ogni fase deve chiudersi con foto, schema, test e nota sugli errori trovati.

Fase	Output	Obiettivo	Criterio di chiusura
H0	Breadboard singolo nodo	Validare sensore/attuazione	Cablaggio visibile e test seriale
H1	Master + NODE_01	Validare comando PC-Master-Nodo	Prima catena funzionante
H2	Master + NODE_01/02/03	Validare stato, sicurezza, docking	Demo minima forte
H3	Tutti i sei nodi	Validare architettura completa	Video demo
H4	Millefori o stripboard	Ridurre cavi e fragilità	Prototipo semi-compatto

H5	Shell 3D grande	Integrare elettronica in forma fisica	Functional prototype 80-120 mm
H6	Mockup nero piccolo	Comunicazione visiva del design finale	24-40 mm non necessariamente elettronico
H7	PCB v0.2	Miniaturizzazione vera	Dopo stabilizzazione completa

17. Checklist hardware di accensione

Area	Controllo	Criterio
Prima alimentazione	Controllare polarità, GND, VCC, corto tra rail	Multimetro prima di USB
ESP32	Caricare sketch blink/serial test	Serial monitor operativo
I2C	Eseguire I2C scanner	Indirizzi OLED/sensori trovati
Servo	Alimentazione 5 V separata e GND comune	Movimento fluido senza reset ESP32
Hall/magneti	Verifica stato digitale senza rimbalzi eccessivi	Stato DOCKED stabile
Proximity	Verifica distanza e soglie	SAFE/WARNING/DANGER coerenti
Camera	Verifica IP/stream separato	STATUS al Master ok
STOP globale	Tutti gli attuatori spenti	Priorità massima

18. Conclusione operativa

L'architettura hardware MicroBot v0.1 è concreta perché non tenta di miniaturizzare subito tutto. La sua forza è separare le funzioni: un Master coordina, i nodi validano comportamento, percezione, aggancio, movimento, telemetria e visione, mentre il PC/controller visualizza e registra gli stati. Questa impostazione produce una piattaforma realmente costruibile entro sei mesi e compatibile con l'evoluzione verso un MicroBot nero compatto, PCB dedicato, docking reale e simulazione sincronizzata.

Formula finale della v0.1

PC Controller -> Master ESP32 -> sei nodi specializzati -> stato reale -> dashboard -> simulazione. Quando questa catena funziona, MicroBot diventa un sistema fisico dimostrabile, non solo un documento o un concept.

Appendice A. Riferimenti interni usati

Fonte interna	Uso nel documento
Manuale operativo dei sei prototipi collegati a un Master	Definisce Master, sei nodi, protocollo, componenti, pinout e checklist.
Documento generale MicroBot / tutto insieme	Definisce architettura, MicroBot come nodo distribuito, elettronica interna, ESP32, MOSFET, Li-Po, coil e safety layer.
Documenti su simulazione e visione	Definiscono connessione tra hardware, dashboard, simulazione e percezione.